TransSGPN: Transistor Network Synthesis via Switching Graph Policy Network

Anonymous submission

Abstract

Transistor network synthesis, finding a compact CMOS switching network that correctly implements a given Boolean function, is a central step in custom-cell design. Exact SAT-based solvers treat every function as an independent search problem: runtime grows exponentially in the number of variables, and nothing learned on one function transfers to the next. We present TransSGPN, which casts synthesis as a sequential graph-generation Markov decision process and trains a Switching Graph Policy Network (SGPN) to solve it. The policy acts on a unified graph that fuses the partial network with a fixed SOP encoding of the target, giving the GNN both existing topology and remaining coverage obligations at every step. Training has three phases: NLL pretraining on known-optimal networks; PPO with a GRPO baseline and a self-tightening reward driving toward minimum transistor count; and joint PDN/PUN synthesis rewarded by the ASTRAN placement objective. On all 254 three-variable functions (508 targets), TransSGPN matches the exact minimum transistor count on 100%, with 97.6% already optimal from pretraining alone, and extends to the four-variable sweep and to hard six-variable non-series-parallel functions. A physical-aware phase then optimizes the joint pull-down/pull-up cell against an ASTRAN placement objective, lowering mean cell placement cost by 18% over count-optimal cells.

1 Introduction

Transistor network synthesis is a foundational step in the design and customization of CMOS standard cells [Clark et al.(2016)Clark, Vashishtha, Shifren et al., Van Cleeff et al.(2020)Van Cleeff, Hougardy, Silvanus et al., Ren and Fojtik(2021)]. Given a Boolean function f, the goal is to build a minimum-transistor switching network—an undirected graph whose edges represent transistors and whose source-to-sink connectivity implements f. The synthesized topology directly determines transistor count, diffusion-sharing opportunities, and downstream layout quality [Callegaro et al.(2010)Callegaro, Marques, Klock et al., Van Cleeff et al.(2020)Van Cleeff, Hougardy, Silvanus et al.]; even a single extra transistor can degrade area, power, and timing after placement and routing, so automatic cell-layout tools depend on a compact transistor network at their input.

Limitations of existing methods. Transistor-network synthesis ranges from heuristic switch-graph construction [Kagaris and Haniotakis(2007), Callegaro et al.(2010)Callegaro, Marques, Klock et al., Kagaris(2016)] to exact SAT-based solvers [Tsai et al.(2025)Tsai, Hsu, Wu et al., Xiao et al.(2023)Xiao, Han, Yang et al.] that can provably find minimum-transistor solutions. All of them, however, solve each function from scratch, with no reuse across instances. Runtime grows exponentially with the number of variables, and the methods offer no generalization to unseen functions. None of these methods *learns* a generalizable synthesis policy, so every new function incurs the full cost of a fresh optimization.

Our approach: learned sequential graph generation. We treat transistor network synthesis as a sequential graph-generation Markov decision process (MDP) and train a policy to solve it. At each step the policy places one transistor, guided by a graph neural network that observes the current partial circuit together with a compact representation of the target function. A single policy trained over a distribution of Boolean functions generalizes at inference time to new functions without any per-instance search. Learning has recently reshaped many stages of the design flow [Huang et al.(2021)Huang, Hu, He et al.], including reinforcement-learning-driven standard-cell layout [Ren and Fojtik(2021)], yet transistor-network synthesis itself has remained the domain of per-instance exact solvers.

Key technical ideas. (1) Theoretical guarantees by construction.

We prove a *Monotonicity* theorem showing that adding transistors never removes existing conducting paths, and derive a *Safety Inheritance* corollary that reduces safety verification at each step from checking the whole network to a single BFS on the new edge. Safety and correctness are therefore maintained as invariants throughout the entire trajectory, not checked post-hoc.

(2) A novel GNN architecture for transistor synthesis. We design TransSGPN, whose core is the Switching Graph Policy Network (SGPN) — a four-head factored policy built on a Message Passing Neural Network

(MPNN) [Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] backbone. Each head specializes in one sub-decision of the action (source node, drain node, control variable, polarity), with safety masks applied independently at each head. A learnable virtual-node embedding allows the policy to propose fresh topology without pre-committing to a node count. All heads share a single MPNN forward pass over a *unified graph* that merges the partial synthesized network with a fixed SOP compensate network.

(3) Three-phase training toward minimum transistor count and physical quality. Phase 1 NLL pretraining on known-optimal solutions initializes the policy. Phase 2 PPO finetuning with a GRPO group-relative baseline, an adaptive self-tightening reward, and an entropy bonus drives convergence toward minimum transistor count without mode collapse. Phase 3 extends the RL loop to joint PDN+PUN synthesis with a physical placement reward from ASTRAN [Ziesemer and Reis(2014)], directly optimizing the weighted placement objective (cell width, gate mismatch, wire length, routing density, and diffusion gaps).

Contributions.

1. **Theoretical foundations for sequential transistor synthesis.** We prove a Monotonicity theorem (adding transistors is order-invariant for conducting paths) and derive a Safety Inheritance corollary that makes safety checking $O(|P^-| \cdot |V|)$ per transistor candidate — independent of the number of already-placed transistors. Together they guarantee that any trajectory produced under our action masks is *correct and safe by construction*, with no post-hoc verification needed (Section 3.1).

2. **A new four-head architecture for transistor network synthesis.** TransSGPN decomposes transistor placement into four factored sub-decisions, each handled by a specialized MLP head sharing an MPNN backbone. Key design choices include: (a) a *unified graph* state that fuses the partial circuit G_t with a fixed SOP compensate network C_g encoding the target function, giving the GNN simultaneous access to current topology and remaining coverage obligations; (b) a *virtual new-node embedding* that extends the node space dynamically, allowing the policy to create fresh internal nodes without an explicit node-count decision; (c) *per-head safety masking* applied independently to the node, variable, and polarity heads, so that the generated action is guaranteed safe without additional filtering; (d) a *batched PackedGraph forward pass* that packs all trajectory steps into one MPNN call, replacing $O(K \cdot T_{\max})$ sequential forward passes with one (Section 3).

3. **Three-phase training: logical and physical optimization.** Phase 1 NLL pretraining initializes the policy on known-optimal solutions. Phase 2 PPO finetuning with a GRPO group-relative baseline, adaptive self-tightening reward $r = T^*(f)/T$, and entropy bonus converges toward minimum transistor count. Phase 3 extends RL to *joint PDN+PUN synthesis* with a physical placement reward, bridging logical synthesis and layout quality in a single learned policy (Section 3).

4. **Physical-aware joint synthesis via ASTRAN reward.** For a target function f, we simultaneously synthesize the pull-down network (PDN, implementing $\neg f(\mathbf{x})$) and pull-up network (PUN, implementing $f(\neg\mathbf{x})$) using the same policy. Each complete PDN+PUN pair is evaluated by ASTRAN [Ziesemer and Reis(2014)] — a standard-cell layout tool — which returns five placement metrics: cell width (CPP), gate mismatches, wire length, routing density, and diffusion cuts. Their weighted sum serves as the joint RL reward, aligning the policy's objective with ASTRAN's own SA optimization (Section 3.6).

5. **Empirical validation.** We evaluate on exhaustive sweeps of all 254 three-variable and $65{,}534$ four-variable Boolean functions, plus a catalog of hard six-variable non-series-parallel functions, using both polarities of each function as separate targets. On the three-variable sweep, TransSGPN reaches the exact minimum transistor count on 100% of the 508 targets, of which 97.6% are already optimal after pretraining; the residual on larger sweeps is closed by Phase 2 (Section 4). Unlike SAT-based exact methods, TransSGPN requires no per-instance solver call at inference.

The remainder of the paper is organized as follows. Section 2 introduces transistor network synthesis and placement/routing background. Section 3 presents our theoretical foundations and the TransSGPN architecture with three-phase training. Section 4 reports experimental results and ablations. Section 5 concludes.

2 Preliminaries

2.1 Transistor Networks

Let $\mathbf{x} = (x_1, \ldots, x_N)$ be N Boolean input variables and $\pi \in \{0,1\}^N$ an input pattern. The *literal alphabet* is $\mathcal{L} = \{x_1, \neg x_1, \ldots, x_N, \neg x_N\}$, and a literal ℓ evaluates to $\ell(\pi) \in \{0,1\}$ under π.

Definition 1 (Transistor Network)**.** A *transistor network* is an undirected multigraph $G = (V, E)$ with two distinguished terminals $\texttt{SRC} \neq \texttt{SNK} \in V$ and edges $E \subseteq \{(u, v, \ell) : u \neq v \in V,\ \ell \in \mathcal{L}\}$, each edge being a transistor controlled by the literal ℓ. Under an input pattern π a transistor is *closed* iff $\ell(\pi) = 1$, and G *conducts* under π iff some $\texttt{SRC}$–$\texttt{SNK}$ path uses only closed transistors. Edges are unordered, so (u, v, ℓ) and (v, u, ℓ) denote the same transistor; the orientation in our action space (Section 3.2) is an artifact of the generation order, not of the network.

Correctness. For a Boolean function $g : \{0,1\}^N \to \{0,1\}$, let $P^+ = g^{-1}(1)$ and $P^- = g^{-1}(0)$ denote its on- and off-patterns. A transistor network G *covers* g if it conducts under every $\pi \in P^+$ (*coverage*), and is *safe* for g if it conducts under no $\pi \in P^-$ (*safety*); it *correctly implements* g iff both hold. We name the two conditions separately because safety applies to *partial* networks as well: every intermediate state of our generative process is safe, whereas coverage is attained only at termination.

2.2 Two Synthesis Problems

Problem A: network synthesis. Given a switching function g, find a smallest transistor network that correctly implements it,

$$\arg\min \{\, |E(G)| \;:\; G \text{ correctly implements } g \,\}.$$

The feasible set is non-empty (Theorem 5), so the minimum is well defined. This is the problem addressed by Phases 1–2 of TransSGPN.

Problem B: cell synthesis. A CMOS standard cell implementing $f(\mathbf{x})$ consists of two complementary networks realized with different transistor types.

Definition 2 (PDN and PUN)**.** The *pull-down network* (PDN) is an NMOS transistor network connected between GND and the output ZN; the *pull-up network* (PUN) is a PMOS transistor network connected between VDD and ZN. The cell is correct iff the PDN pulls ZN low whenever $f = 0$ and the PUN pulls ZN high whenever $f = 1$.

Problem B is not a new problem: it decomposes into two instances of Problem A.

Proposition 1 (Dual Switching Network Equivalence)**.** *Let P^+ and P^- denote the on- and off-patterns of the cell function f. A CMOS cell correctly implements $f(\mathbf{x})$ if and only if its PDN correctly implements $\neg f(\mathbf{x})$, with on-patterns P^-, and its PUN correctly implements $f(\neg\mathbf{x})$, with on-patterns $\{\neg\pi : \pi \in P^+\}$.*

The proof, together with a worked AND2 decomposition, is given in the supplementary material (App. A–B). Proposition 1 is what allows a *single* policy to serve the whole framework: the PDN and PUN are two instances of Problem A that share the same Boolean structure and differ only in their on/off pattern sets, so a policy conditioned on those sets is agnostic to transistor type. This is the foundation for the joint PDN+PUN training of Phase 3 (Section 3.6).

2.3 Physical Evaluation of a Placed Cell

A solved instance of Problem B is not yet a layout. Its transistors must be placed into two rows, and because adjacent devices share a diffusion region only when their abutting terminals carry the same net, the *ordering* within each row, and not only the device count, determines the area the cell occupies. Quality is therefore measured on the placed cell by five metrics: cell width in contacted poly pitch (CPP) units, gate mismatches between the P and N rows, wire length, routing density, and diffusion cuts. Definitions of the five metrics are given in App. C. We use ASTRAN [Ziesemer and Reis(2014)], a standard-cell layout tool, to place each synthesized PDN+PUN pair and to report these quantities; their weighted sum Φ (Eq. 3) is the objective Phase 3 of TransSGPN minimizes directly as the RL reward (Section 3.6).

3 The TransSGPN Framework with Switching Graph Policy Network

3.1 Theoretical Foundations

We establish five results that jointly guarantee that safety-masked sequential generation is *sound* (every network it produces is valid) and *complete* (every valid network remains reachable). Write $\mathrm{Paths}(G)$ for the set of pairs (π, P) such that path P conducts $\texttt{SRC} \to \texttt{SNK}$ in G under pattern π. Proofs and worked 2-variable examples are given in the supplementary material (App. A–B).

Theorem 2 (Monotonicity of Conducting Paths)**.** *For any transistor network G and any transistor $e \notin E(G)$, $\mathrm{Paths}(G) \subseteq \mathrm{Paths}(G \cup \{e\})$.*

Monotonicity has two consequences, one for each direction of edge modification.

Corollary 3 (Incremental Safety Checking)**.** *If G is safe for g, then $G \cup \{(u, v, \ell)\}$ is safe for g iff (u, v, ℓ) alone creates no $\texttt{SRC} \to \texttt{SNK}$ path under any $\pi \in P^-$.*

Lemma 4 (Deletion Preserves Safety)**.** *If $G' \subseteq G$ and G is safe for g, then G' is safe for g.*

Corollary 3 reduces the per-candidate safety test from $O(|E|)$ to $O(|P^-| \cdot |V|)$, independent of how many transistors are already placed. This is what makes the safety mask affordable at every step. Lemma 4 is the mirror statement: deletion can only lower conduction, so the network reduction of Section 3.2 need re-verify coverage only, never safety.

Theorem 5 (SOP Network Correctness)**.** *Let $G_{\mathrm{SOP}}(g)$ be the series-parallel network with one N-transistor series chain per $\pi \in P^+$, all sharing $\texttt{SRC}$ and $\texttt{SNK}$. Then $G_{\mathrm{SOP}}(g)$ correctly implements g.*

Theorem 5 both guarantees that the feasible set of Problem A is non-empty and supplies the fixed *compensate network* that conditions our state representation (Section 3.2).

Theorem 6 (Completeness of Safety-Masked Search)**.** *For any valid G^* implementing g, every permutation of $E(G^*)$ is a valid trajectory under the safety mask.*

The mask therefore never prunes a valid solution. It forbids only transistors that immediately create an off-pattern path, and such a transistor cannot appear in any correct network; the policy searches the full space of valid partial subnetworks.

3.2 MDP and Unified Graph State

State. The state at step t is the *unified graph* $H_t = G_t \cup C_g$, where $G_t = (V_t, E_t)$ is the partial synthesized network and $C_g = G_{\mathrm{SOP}}(g)$ is the fixed compensate network of Theorem 5, held constant throughout the episode. The two components share only SRC and SNK; all internal nodes are disjoint. This gives the GNN simultaneous access to the topology built so far and to the full logical specification of g, so the policy can reason jointly about what exists and which coverage obligations remain. Nodes carry their type (SRC, SNK, G-internal, C-internal), their degree in G_t and in C_g separately, and a coverage indicator; edges carry component membership, variable index, and polarity (dimensions in App. C). The node space is extended to $V_t^+ = V_t \cup \{v_{\mathrm{new}}\}$ by a single learnable embedding $\mathbf{e}_{\mathrm{new}} \in \mathbb{R}^d$, which lets the policy open a fresh internal node at any step without a separate node-count decision and keeps the action space shape independent of how many nodes exist.

Action and masks. An action $a_t = (u, v, \ell)$ adds one transistor, where the literal $\ell = (k, s)$ is given by a variable index k and a polarity s. Two masks apply. The **reachability mask** prunes nodes u that are structurally disconnected, and the **safety mask** forbids any (u, v, ℓ) that creates a P^--path (Corollary 3). By Theorem 6 the safety mask is complete, so masking removes no reachable solution.

Termination. Because safety is an invariant of every masked trajectory, coverage is the only remaining condition for correctness: an episode terminates successfully as soon as G_t covers g, at which point G_t correctly implements g by construction. Episodes that have not covered g within a budget of $T_{\max}$ placements terminate as incomplete.

Network reduction. A terminal network may contain devices that do not affect the function it computes, and counting them overstates its cost. A transistor is *structurally dead* if it lies on no simple SRC $\rightarrow$ SNK path; such devices carry no current and are removed by keeping only edges whose biconnected block lies on the block-cut-tree path between the rails. More generally, a network is *irredundant* if no single transistor can be deleted while still covering every on-pattern, and we delete greedily to a fixed point. By Lemma 4 neither step can introduce an off-pattern path, so only coverage is re-verified. We write T_{eff} for the reduced size and use it both in the reward and when reporting results, so the policy is never penalized for redundancy a downstream pass would remove. The effect is large: on the 6-input NSP benchmark of Section 4, reduction lowers mean generated size from 41.4 to 19.0 transistors.

Reward. The reward is terminal, undiscounted, and identical at every step of the trajectory:

$$r = \begin{cases} T^*(g)/T_{\text{eff}} & \text{Phase 2, complete network} \\ \Phi^*(f)/\Phi & \text{Phase 3, complete PDN+PUN pair} \\ 0 & \text{incomplete,} \end{cases} \tag{1}$$

where $T^*(g)$ and $\Phi^*(f)$ are the best values found so far for that target. $T^*(g)$ is initialized by the first complete network found for g. This is an *adaptive self-tightening* baseline: the standard rises automatically whenever the policy discovers a better solution.

3.3 Switching Graph Policy Network: Factored Policy

The SGPN encodes H_t with an MPNN backbone [Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] of L message-passing layers and hidden dimension d, followed by a self-attention block over the nodes of each graph, yielding node features $\mathbf{h}_i \in \mathbb{R}^d$ and a graph summary $\mathbf{g} \in \mathbb{R}^{2d}$. Four MLP heads decode the action sequentially, each conditioned on the choices already sampled:

$$\begin{aligned} s_u &= \text{MLP}_1([\mathbf{h}_u \,\|\, \mathbf{g}]), \\ s_v &= \text{MLP}_2([\mathbf{h}_{u^*} \,\|\, \mathbf{h}_v \,\|\, \mathbf{g}]), \\ \mathbf{s}_{\text{var}} &= \text{MLP}_3([\mathbf{h}_{u^*} \,\|\, \mathbf{h}_{v^*}]), \\ \mathbf{s}_{\text{sgn}} &= \text{MLP}_4([\mathbf{h}_{u^*} \,\|\, \mathbf{h}_{v^*}]). \end{aligned}$$

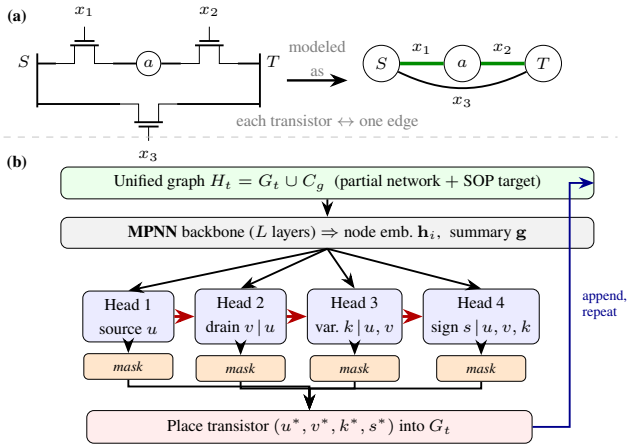

Figure 1: **(a)** A transistor network is a switching graph: an input pattern yields a conducting source-to-sink ($S \rightarrow T$) path exactly on the on-patterns of the target (coverage), and no path on the off-patterns (safety). **(b)** The Switching Graph Policy Network (SGPN) synthesizes such a network one transistor at a time. A shared MPNN backbone encodes the unified graph $H_t = G_t \cup C_g$ (the partial network fused with the fixed SOP target); the four heads then fire *sequentially*—source u, drain v, variable k, sign s—each conditioned on the earlier selections and its own safety mask. The placed transistor (u^*, v^*, k^*, s^*) is appended to G_t and the step repeats until the network covers the target.

Head 1 scores every $u \in V_t^+$ as the first endpoint and is masked to the reachability set; Head 2 scores each $v \neq u^*$ and is masked to exclude nodes admitting no safe literal for (u^*, v); Heads 3 and 4 share their input and select the control variable and its polarity, masked to the variables and signs that keep (u^*, v^*, ℓ) safe. The four decisions factor as

$$\begin{aligned} \log \pi(a \mid H_t) = \log p_1(u) + \log p_2(v \mid u) \\ + \log p_3(k \mid u, v) + \log p_4(s \mid u, v, k), \end{aligned} \tag{2}$$

where p_j is the masked softmax of Head j over its safe candidates. By Corollary 3, every action sampled from this distribution preserves the safety invariant, so no post-hoc filtering is required.

Batched forward pass. A trajectory of length T under K rollouts would naively need $O(K \cdot T_{\max})$ separate MPNN calls. Instead we pack every step graph across all trajectories and all functions in the mini-batch into a single PackedGraph and encode them with *one* MPNN forward pass, slicing per-step node features from the shared output for the head MLPs. This applies to both pretraining and PPO, and is what makes large-batch training feasible on a single GPU.

3.4 Phase 1: NLL Pretraining

We collect a dataset $\mathcal{D}$ of known-optimal transistor networks, each paired with its target function, and replay every net-

work as a sequence of placements to obtain per-step labels (u, v, k, s).

Construction order. A transistor network is an *unordered* set of devices, so replaying it as a sequence requires choosing a linearization $e_1, \ldots, e_{|E^*|}$, and that choice defines the learning problem. The natural default is to order devices by node index, equivalently by a breadth-first traversal from the rails, but this makes the first-endpoint label an artifact of the indexing convention rather than of the circuit. We instead use a *path (ear) decomposition*: the sequence is emitted as successive walks, where the first walk starts at `SRC` and extends through unvisited nodes until it closes into already-visited structure, preferentially at `SNK`, and each later walk (an *ear*) starts at an already-visited node and extends until it reconnects. Every emitted action then satisfies the invariant that u is already present in the partial network, so the walk direction fixes the orientation.

This aligns supervision with the semantics of the task. Conduction is achieved only when a `SRC` $\rightarrow$ `SNK` path *closes*, so decomposing the target into completed paths makes each label continue a coherent sub-goal. It also changes the difficulty profile: during a walk extension the partial network has exactly one dangling endpoint, and that endpoint *is* the next u, so Head 1 becomes structurally determined rather than conventional. The residual difficulty concentrates at *ear starts*, where the model must choose which existing node to branch from. That is precisely the structure-sharing decision governing compactness. Section 4 shows this single change raises source-selection accuracy by roughly 30 points, far more than any architectural variation we tested.

Label consistency. A function typically admits many optimal or near-optimal networks, and retaining several implementations of the *same* function pairs identical states with conflicting targets, giving the objective an irreducible floor. We therefore keep a single implementation per function, the most compact one, which removes the conflict and matches the downstream objective of minimizing transistor count.

The objective is the mean negative log-likelihood of the reference actions,

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{|\mathcal{D}|} \sum_{(G^*, g) \in \mathcal{D}} \sum_{t=1}^{|E^*|} \log \pi(a_t^* \mid H_{t-1}),$$

where H_{t-1} is built from the prefix G_{t-1}^* and the fixed compensate network C_g. We train with Adam until convergence.

3.5 Phase 2: PPO Finetuning

We finetune the pretrained policy with PPO. A frozen behaviour policy π_{old} generates trajectories and the live policy recomputes log-probabilities for the clipped surrogate loss. We resynchronize $\pi_{\mathrm{old}} \leftarrow \pi_\theta$ every M updates, which keeps the importance ratio π/π_{old} within a controlled range.

GRPO baseline. The reward is the terminal, undiscounted $r = T^*(g)/T_{\mathrm{eff}}$ of Eq. 1. For each function we sample K trajectories and normalize the advantage within the group,

$$\hat{A}_k = \frac{r_k - \bar{r}}{\sigma_r + \varepsilon},$$

where $\bar{r}$ and σ_r are the mean and standard deviation of $\{r_k\}_{k=1}^K$. This group-relative baseline [Shao et al.(2024)Shao, Wang, Zhu et al.] needs no value network and gives a positive signal to any trajectory shorter than its group average.

Entropy bonus. Because the reward tightens as $T^*(g)$ improves, the group can collapse onto a single solution and stop exploring. We add an entropy term $\mathcal{L} = \mathcal{L}_{\mathrm{PPO}} - \lambda_H \bar{H}$, where $\bar{H}$ is the mean per-step policy entropy across the four heads, so the policy continues to sample circuits shorter than its current best.

3.6 Phase 3: Physical-Aware Joint PDN+PUN Synthesis

Phases 1 and 2 optimize a single switching network. Phase 3 initializes from the Phase 2 checkpoint and extends the RL loop to a complete CMOS cell, scored by physical placement quality rather than transistor count.

Joint synthesis. By Proposition 1, the PDN and PUN of a cell for f are two instances of Problem A, with on-patterns P^- and $\{\neg\pi : \pi \in P^+\}$ respectively. The *same* policy synthesizes both: it is conditioned only on the on/off pattern sets and is agnostic to transistor type.

Physical reward. Each complete pair is emitted as a SPICE `.subckt` (App. C) and placed by ASTRAN [Ziesemer and Reis(2014)], which reports the five metrics of Section 2.3. We score the pair by ASTRAN's own weighted cost,

$$\begin{aligned}\Phi = w_c W_{\mathrm{cpp}} + w_m N_{\mathrm{gm}} + w_w W_{\mathrm{wl}} \\ + w_d D_{\mathrm{rt}} + w_g N_{\mathrm{gap}},\end{aligned} \tag{3}$$

with $(w_c, w_m, w_w, w_d, w_g) = (3, 4, 1, 4, 2)$ matching the ASTRAN simulated-annealing cost function exactly, so minimizing Φ is equivalent to minimizing ASTRAN's internal objective.

Paired trajectories. For each function we generate K paired trajectories $(\tau_k^{\mathrm{PDN}}, \tau_k^{\mathrm{PUN}})_{k=1}^K$ and call ASTRAN only on complete pairs, parallelized over a thread pool. Rewards $r_k = \Phi^*(f)/\Phi_k$ follow Eq. 1, and GRPO normalizes them within the K pairs. The resulting $\hat{A}_k$ is applied to *every* step of both trajectories: a pair that places well reinforces both networks, a pair that places poorly penalizes both. This is the only structural difference from the Phase 2 loop.

4 Experiments

4.1 Setup

Dataset. We evaluate on exhaustive SAT-based sweeps at three input widths. For $N = 3$, the sweep covers all 254 non-trivial functions; for $N = 4$ it covers $65{,}534$ functions; and for $N = 6$ we use a catalog of 53 non-series-parallel (NSP) functions, which are the hardest instances because they admit no series-parallel realization. Each sweep enumerates, per function, one synthesis attempt per transistor budget, recording the realized network when the budget is satisfiable; the smallest satisfiable budget is that function's exact

Table 1: Benchmark scale. Each function yields a PDN and a PUN target.

Benchmark	Functions	Targets	Optimum
$N = 3$ sweep	254	508	1–8
$N = 4$ sweep	65,534	131,068	4–12
$N = 6$ NSP	53	53	5–7

optimum. Every function contributes *two* synthesis targets: the pull-down network realizing $\neg f$ and the pull-up network realizing $f(\neg x)$. Reference counts for the pull-up target are obtained by matching the dual function $f(\neg x)$ back to its own entry in the sweep. This yields 508 targets at $N = 3$ and $131{,}068$ at $N = 4$. Because the sweeps are exhaustive, evaluation is against exact optima rather than a held-out split; we additionally freeze a random subset of 256 solved 4-input targets as a fixed set for the Phase 3 comparison.

Implementation. TransSGPN uses an MPNN backbone with $d = 128$, $L = 3$ layers, and MLP hidden dimension 256, followed by a self-attention block over the nodes of each graph before the policy heads. Phase 1 uses the path-order supervision of Section 3.4 with one implementation per function, trained for 200 epochs with Adam (cosine-annealed lr $2 \times 10^{-3} \rightarrow 10^{-4}$) and a $2\times$ weight on the Head-1 term. Phase 2 uses $K = 16$ trajectories per update, lr 10^{-4}, $\varepsilon = 0.2$, $\lambda_H = 0.05$, up to 200 updates per target with early stopping when the exact optimum is reached, and two independent restarts from the pretrained checkpoint. Reported transistor counts are the reduced counts T_{eff} of Section 3.2. Phase 3 initializes from the Phase 2 checkpoint, generates paired PDN+PUN trajectories per function, and evaluates complete pairs with ASTRAN (freePDK45, SA quality 1, weights $(3, 4, 1, 4, 2)$). All experiments run on a single NVIDIA H100 GPU.

Baselines. We compare against:

- **MiniTNtk** [Xiao et al.(2023)Xiao, Han, Yang et al.]: exact SAT-based count minimization (source of our reference optima).
- **TransSGPN Phase 1**: NLL pretraining only.
- **TransSGPN Phase 2**: transistor-count RL (single network).

For physical metrics (Phases 2–3), we generate the SPICE netlist for each synthesized PDN+PUN pair and evaluate it with ASTRAN using the same placement parameters to ensure a fair comparison.

Metrics. *Logical metrics* (Phases 1–2):

- **Success rate**: fraction of functions with a correct implementation.
- **Optimality rate**: fraction matching the known minimum transistor count.
- **Avg. transistors**: mean transistor count over correct solutions.

Physical metrics (Phase 3):

Table 2: Optimality rate (fraction of targets reaching the exact SAT optimum). Rates marked † (N=4, N=6) are over the evaluated subset of each sweep.

Method	N=3 **(508)**	N=4	N=6 **NSP**
MiniTNtk [Xiao et al.(2023)Xiao, Han, Yang et al.]	100	100	100
TransSGPN Phase 1	97.6	49.2	0
TransSGPN Phase 2 (ours)	**100**	88†	24.5†

- **ASTRAN objective** Φ (Eq. 3): weighted sum of CPP, gate mismatches, WL, routing density, and diffusion cuts.
- **CPP** (cell width): total poly columns used by the placed cell.
- **Gate mismatches**: positions where P-row and N-row gate signals differ.
- **Nr. Gaps**: number of diffusion cuts (Euler-path breaks).
- **WL / Rt. Density**: wire length and peak routing congestion.

4.2 Main Results

Table 2 reports optimality rates for the pretrained policy (Phase 1) and after transistor-count finetuning (Phase 2). On the exhaustive 3-input sweep, TransSGPN attains the exact optimum on **all** 508 **targets**. The decomposition is informative: the pretrained policy alone is already optimal on 97.6% of targets at its first sampled evaluation, and Phase 2 closes the remaining 12 targets, each within 12 updates. This indicates that when supervision is consistent and ordered along conduction paths, imitation alone recovers exact-optimal synthesis at this width, and reinforcement learning is needed only for a small residual.

At $N = 4$ the residual is substantially larger and Phase 2 carries more of the load; we report the optimality rate over the evaluated subset of targets and leave the full four-input sweep to future work. At $N = 6$ the two phases separate cleanly: the pretrained policy produces a *correct* network for 50 of 53 NSP functions but at a mean reduced cost of 19.0 transistors against optima near 7, so completion is largely solved by pretraining while compaction is the task left to Phase 2.

MiniTNtk is an exact SAT-based solver that supplies the reference optima and therefore attains 100% by construction; its cost is a per-instance solver invocation, whereas TransSGPN amortizes synthesis into a learned policy and needs no solver at inference, matching the exact optimum on all 508 three-input targets. Daggers (†) mark rates over the evaluated subset of the N=4 and N=6 sweeps.

4.3 Ablation: Supervision Order Dominates Architecture

Our central design claim is that *how* a reference network is linearized into a supervision sequence matters more than model capacity. Table 3 varies the two factors independently at $N = 4$. Under index/breadth-first order, Head-1 accuracy saturates near 0.54–0.63 regardless of capacity; under the path order of Section 3.4 it exceeds 0.88 with the *same* encoders. The ordering is worth roughly $+30$ points, while doubling width and adding attention is worth 4–8.

Table 3: Head-1 (source-selection) accuracy at N=4: architecture × supervision order.

Architecture	Index/BFS order	Path order
d=64, MLP 128	0.543	0.886
d=128, MLP 256, attention	0.625	**0.932**

The effect is not specific to $N = 4$: at $N = 6$ the same substitution raises Head-1 accuracy from 0.505 to 0.91. We attribute this to the structural argument of Section 3.4: under the path order the source is determined by the graph during walk extensions, whereas under an index order it is a convention the model must memorize.

Effect of label consistency. Retaining every implementation of a function introduces conflicting targets for identical states. At $N = 6$, restricting supervision to one implementation per function raises Head-1 accuracy from 0.443 to 0.505; in the multi-implementation setting the loss also degrades over training rather than continuing to descend, consistent with an irreducible label-conflict floor.

Effect of network reduction. Because redundant devices are removable in post-processing (Lemma 4), scoring unreduced networks penalizes the policy for structure a downstream pass deletes. On the $N = 6$ NSP benchmark, mean generated size falls from 41.4 (unreduced) to 40.1 (structural reduction only) to 19.0 (full irredundant reduction), and one function reaches its exact optimum under reduction alone.

Effect of the search budget. Completion depends strongly on the rollout step budget: at $N = 6$, raising the budget from 40 to 80 placements raises the fraction of functions for which a correct network is ever found from 13/36 to 30/36, while mean size grows, confirming that under a tight budget the policy fails to complete rather than failing to compact.

4.4 Analysis

Top-k behaviour of the pretrained policy. Although top-1 Head-1 accuracy is 0.50 on 6-input states, the correct node lies in the top 3 candidates 78% of the time and in the top 5 89% of the time (mean rank 1.6 of ∼64). Since Phase 2 *samples* from the policy rather than taking its argmax, top-k mass — not top-1 accuracy — is the relevant measure of prior quality, which explains why a policy with 0.50 top-1 accuracy still finetunes effectively.

Where source selection fails. Accuracy is 0.62 on small partial networks but falls to 0.36 on mid-sized ones, and is markedly lower when the target is an interior node (0.37) than a rail (0.66), with the model over-predicting rails. Errors are largely not near-misses and the model is confidently wrong rather than undecided, indicating a learning rather than a tie-breaking failure that richer per-node structural features could address.

Representational ceiling. A Weisfeiler–Leman refinement over the unified graph states assigns every candidate node a distinct colour within two rounds, so the achievable Head-1 accuracy is 1.0. The observed gap is therefore an optimization and inductive-bias limitation, not a limitation of the state encoding.

Table 4: Physical placement cost $\Phi_{\mathrm{plc}} = \mathrm{CPP}+\mathrm{GM}+\mathrm{Gaps}$ (lower is better): MiniTNtk's count-optimal cells vs. our physically-optimized cells. Three-input covers all 254 cells; four-input covers the 128 cells for which Phase 3 produces a placement-improved cell.

Method	3-input (254)	4-input (128)
MiniTNtk (count-optimal)	12.74	19.80
TransSGPN Phase 3 (ours)	**10.44**	**16.82**

Cost of the reward variants. All Phase-2 reward variants cost essentially the same per update (≈ 1.6 s), since rollout generation dominates while the baseline and exploration terms are nearly free. Throughput is instead governed by device sharing—per-update time grows from 1.9 s at one process per GPU to 49 s at sixteen, as each step issues many small kernels that serialize—so variant comparisons are meaningful only at matched concurrency.

4.5 Phase 3: Physical-Aware Placement

Phase 3 finetunes the joint PDN+PUN policy against the ASTRAN placement objective (Section 3.6). A target is now a *function*: the objective exists only once *both* networks complete, since ASTRAN places the combined cell, and the reward is assigned jointly to the two trajectories. We ask whether optimizing for placement — rather than transistor count alone — yields cells that lay out more compactly.

Table 4 compares, over all 254 three-input functions, the placement cost of **MiniTNtk**'s count-optimal networks — the reference solutions our own counts are measured against — with our Phase-3 physically-optimized networks, scored by ASTRAN's exact placement metrics: cell width (CPP), gate mismatches (GM), and diffusion gaps, summed as Φ_{plc}. MiniTNtk minimizes transistor count only; Phase 3 instead selects, among the many count-optimal realizations of a function, those that also place compactly, lowering mean placed-cell cost by 18% (3-input, $12.74 \rightarrow 10.44$) and 15% (4-input) while still matching the optimal transistor count on 95% of the four-input subset. Transistor count and layout quality are thus distinct objectives.

A routing-aware objective (wire length and routing density under real ASTRAN routing) is left to future work.

5 Conclusion

We presented TransSGPN, which casts transistor network synthesis as a safety-masked sequential graph-generation MDP and solves it with a four-head policy over a unified graph, correct and safe by construction under per-head masks. Three-phase training—NLL pretraining, GRPO-based transistor-count RL, and a physical-aware ASTRAN-reward phase—matches the exact optimum on all 508 three-variable targets, scales to four and six variables, and cuts mean placement cost 18%/15% below count-optimal cells—with no per-instance solver call at inference.

References

[Callegaro et al.(2010)Callegaro, Marques, Klock et al.] Callegaro, V.; Marques, F. d. S.; Klock, C. E.; et al. 2010. SwitchCraft: A Framework for Transistor Network Design. In *Symposium on Integrated Circuits and Systems Design (SBCCI)*, 49–53.

[Clark et al.(2016)Clark, Vashishtha, Shifren et al.] Clark, L. T.; Vashishtha, V.; Shifren, L.; et al. 2016. ASAP7: A 7-nm FinFET Predictive Process Design Kit. *Microelectronics Journal*, 53: 105–115.

[Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] Gilmer, J.; Schütt, K. T.; Mayr, A.; et al. 2017. Neural Message Passing for Quantum Chemistry. *International Conference on Machine Learning (ICML)*.

[Huang et al.(2021)Huang, Hu, He et al.] Huang, G.; Hu, J.; He, Y.; et al. 2021. Machine Learning for Electronic Design Automation: A Survey. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 26(5): 1–46.

[Kagaris(2016)] Kagaris, D. 2016. MOTO-X: A Multiple-Output Transistor-Level Synthesis CAD Tool. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 35(1): 114–127.

[Kagaris and Haniotakis(2007)] Kagaris, D.; and Haniotakis, T. 2007. A Methodology for Transistor-Efficient Supergate Design. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 15(4): 488–492.

[Ren and Fojtik(2021)] Ren, H.; and Fojtik, M. 2021. NV-Cell: Standard Cell Layout in Advanced Technology Nodes with Reinforcement Learning. In *ACM/IEEE Design Automation Conference (DAC)*, 1291–1294.

[Shao et al.(2024)Shao, Wang, Zhu et al.] Shao, Z.; Wang, P.; Zhu, Q.; et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

[Tsai et al.(2025)Tsai, Hsu, Wu et al.] Tsai, J.-C.; Hsu, W.-M.; Wu, K.-L.; et al. 2025. MuSTNet: SAT-based Exact Multi-Stage Transistor Network Synthesis with Placement Awareness. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 1–8.

[Van Cleeff et al.(2020)Van Cleeff, Hougardy, Silvanus et al.] Van Cleeff, P.; Hougardy, S.; Silvanus, J.; et al. 2020. BonnCell: Automatic Cell Layout in the 7-nm Era. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(10): 2872–2885.

[Xiao et al.(2023)Xiao, Han, Yang et al.] Xiao, W.; Han, S.; Yang, Y.; et al. 2023. MiniTNtk: An Exact Synthesis-based Method for Minimizing Transistor Network. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.

[Ziesemer and Reis(2014)] Ziesemer, A.; and Reis, R. 2014. ASTRAN: Automatic Synthesis of Transistor Networks. In *IEEE International Symposium on Integrated Circuits and Systems (ISICAS)*. Open-source standard-cell layout framework; `https://github.com/aziesemer/astran`.